

Gaussian Component-wise Boosting

Luca Messerschmidt Bjarne Herbst Benedict B. Schnitzler

2026-05-21

Overview

This vignette introduces the basic workflow for fitting Gaussian component-wise boosting models using **asp26boost**.

Boosting is a stage-wise optimization procedure that combines many simple model updates into a flexible prediction model. From a statistical perspective, boosting can be viewed as an iterative empirical risk minimization algorithm. Starting from an initial estimate, the model is repeatedly improved by fitting simple base learners to the current residual information and updating only the best-performing model component at each iteration. Early stopping acts as a form of regularization and often leads to sparse models by selecting only a subset of available predictors.

For Gaussian response models, component-wise boosting minimizes the negative log-likelihood. Beginning with an intercept-only model, each predictor is fitted separately to the current residuals. The predictor yielding the largest reduction is selected and updated by a shrunk step determined by the learning rate. Repeating this procedure yields coefficient estimates, variable selection, and regularization simultaneously. In the implementation used here, separate predictor sets can be specified for the mean (μ) and scale (σ) components via the arguments X and Z .

The implementation provided by **asp26boost** follows this component-wise boosting framework and offers methods for model fitting, prediction, summarization, and visualization.

Installation

```
devtools::install_gitlab(  
  repo = "bjarne.herbst01/asp26boost",  
  host = "https://gitlab.gwdg.de"  
)
```

Simulated Sample Data

To illustrate the basic functionality, we generate a synthetic regression dataset with a sparse underlying signal.

```
set.seed(27092001)  
  
n <- 1000 # number of observations  
p <- 10   # number of covariates  
  
X <- matrix(  
  rnorm(n * p),  
  nrow = n,  
  ncol = p  
)
```

```

colnames(X) <- paste0("x", seq_len(p))

# Heteroskedastic scale model using the same three predictors
beta_mu <- c(2, -1.5, 1, rep(0, p - 3))
mu <- as.numeric(X %*% beta_mu)

beta_sigma <- c(0.4, -0.3, 0.2, rep(0, p - 3))
sigma <- exp(as.numeric(X %*% beta_sigma) / 2)

y <- rnorm(n, mean = mu, sd = sigma)

sample_data <- data.frame(
  y = as.numeric(y),
  X
)

head(sample_data)
>      y      x1      x2      x3      x4      x5
> 1  2.0879843 -0.4261948 -1.1171738  0.48469532  1.2131425 -0.336170941
> 2 -0.8686132 -0.7131972  0.4245545  0.44002122 -0.8269670 -0.942629273
> 3 -5.1103592 -1.8385995  0.3599087 -0.22333695 -0.4738789 -0.996149133
> 4 -1.8438897  0.8625711  1.5398105 -0.07525713  1.4213003 -0.003867292
> 5 -3.4377866 -1.0810953  1.3918728  0.35499283 -0.5583812  0.978657121
> 6 -4.1332283 -1.7623401 -0.8189664 -1.73188716 -1.5627475 -0.983039525
>      x6      x7      x8      x9      x10
> 1 -1.2355394  0.70395175  2.1222730 -1.7657446  0.4977308
> 2  0.8983855 -1.19335497 -0.9879921  0.2926119  0.8580337
> 3 -2.0669550 -0.65491769  0.5644572  0.5134706  0.3168015
> 4  0.4912211  0.22194407  0.7515952 -1.4363766  0.8185884
> 5 -0.7280939  0.04176014  0.3786812 -0.3892265 -1.1157866
> 6 -0.8300621 -2.22611065  0.9980038  0.3107517 -0.5029159

```

The response depends only on the first three predictors. The remaining covariates contain noise and are included to illustrate the variable-selection behavior of component-wise boosting.

Model Fitting

Gaussian component-wise boosting models are fitted using `boost_gaussian()`.

```

fit <- boost_gaussian(
  X = sample_data[, -1], # predictors for mu
  Z = sample_data[, -1], # predictors for sigma
  y = sample_data$y,
  mstop = 100,
  nu_mu = 0.05,
  nu_sigma = 0.05
)

```

The most important tuning parameters are:

- `mstop`: number of boosting iterations.
- `nu_*`: learning rate controlling the size of each boosting update, defined separately for the mean and scale.

Internally, predictors are standardized before fitting. The model is initialized with the mean response and an initial

scale estimate and updated iteratively by selecting the predictor that provides the largest reduction in squared-error loss.

Printing the fitted object provides a compact overview of the model.

```
fit
>
> Distributional Gaussian Gradient Boosting
> -----
> Call:
> boost_gaussian(X = sample_data[, -1], Z = sample_data[, -1], y = sample_data$y,
  ↪ mstop = 100, nu_mu = 0.05, nu_sigma = 0.05)
>
> Tuning parameters:
> method      : cyclic
> mstop       : 100
> nu_mu       : 0.05
> nu_sigma    : 0.05
>
> Location predictors (X):
> Total              : 10
> Selected (>= 1 time) : 2
> Never selected (coef=0): 8
>
> Selected variables and selection counts:
>
> x1 x2
> 77 23
>
> Scale predictors (Z):
> Total              : 10
> Selected (>= 1 time) : 10
> Never selected (coef=0): 0
>
> Selected variables and selection counts:
>
> x8 x9 x2 x4 x7 x1 x10 x5 x6 x3
> 21 17 16 11 11 6 6 6 5 1
>
> Training fit (location):
> RSS      : 5052.613
> MSE      : 5.0526
> R-squared : 0.4158
```

Typical output includes:

- the original model call
- the tuning parameters specified during the model call
- The number of predictors, including which were (not) selected
- basic training-fit statistics
- the names of the selected variables and selection frequencies

Model Summary

A more detailed summary can be obtained using `summary()`.

```

summary(fit)
>
> Distributional Gaussian Gradient Boosting - Model Summary
> -----
> Call:
> boost_gaussian(X = sample_data[, -1], Z = sample_data[, -1], y = sample_data$y,
+ mstop = 100, nu_mu = 0.05, nu_sigma = 0.05)
>
> Dimensions : n = 1000
>               pX = 10 (location predictors) 2 selected, 8 zero
>               pZ = 10 (scale predictors)    10 selected, 0 zero
>
> Iterations : method = cyclic mstop = 100 nu_mu = 0.05 nu_sigma = 0.05
>
> Residuals (y - mu_hat):
>      Min      1Q   Median     Mean      3Q      Max
> -8.6367 -1.449 -0.0827  0.0151  1.385  8.3161
>
> RSS : 5052.6133    MSE : 5.0526    R-squared : 0.4158
>
> Location submodel coefficients (original predictor scale):
> Variable              Coefficient
> -----
> (Intercept)          -0.0302
> x1                    0.8738
> x2                   -0.2668
>
> Variables never selected (zero coefficient):
> x3, x4, x5, x6, x7, x8, x9, x10
>
> Scale submodel coefficients
> Linear predictor : log(sigma) [original predictor scale]
> Interpretation   : one unit increase in z_j multiplies sigma
>                   by exp(coefficient)
> Variable              Coefficient
> -----
> (Intercept log-sigma) 0.8339
> x8                    0.0255
> x9                   -0.0227
> x4                    0.0199
> x7                    0.0179
> x2                   -0.0161
> x5                    0.0074
> x10                  -0.0069
> x6                    0.0048
> x1                    0.0036
> x3                   -5e-04

```

The summary object provides additional information on:

- summary of residuals
- coefficient estimates on the original predictor scale for selected predictors
- list of non-selected predictors (coefficient = 0)

Prediction

Predictions can be computed using `predict()`.

```
predict(fit)
> [1] -0.104620244 -0.766681352 -1.732868414 0.312801662 -1.346206136
> [6] -1.351757945 -0.027102905 1.582982836 -0.103570063 -0.893195223
> [11] -1.721875572 0.707034326 1.071599728 -0.745674398 -1.847541232
> [16] -0.294509369 -1.241001233 0.024115192 0.291381373 0.720592466
> [21] -1.068787979 0.631082250 -0.264623768 -0.683110453 -0.087277164
> [26] -0.406614395 -0.569889767 1.962081391 -0.991217355 -1.621168087
> [31] -0.488540204 -1.517595352 -0.330917386 0.067260827 -0.132560270
> [36] 0.510694908 0.568182666 -0.759754960 2.142414383 0.942517316
> [41] 1.584275598 0.449805692 -0.654025296 0.038211917 -0.500747379
> [46] 0.185102702 -0.469038155 -0.779451866 0.068492335 -0.378450501
> [51] -0.983039136 0.107989251 1.963230166 0.578157554 0.837569610
> [56] -0.698275772 -0.432462725 1.079911047 0.082119780 0.088966766
> [61] -2.651145596 0.153330515 0.041113175 -1.483597338 -1.021002962
> [66] -1.046769869 0.912083540 -0.762921663 0.137372048 0.119009501
> [71] -1.013663503 1.255262710 0.601149521 -1.632650586 0.826308426
> [76] 1.572206455 0.309178563 0.574968899 -1.756609738 -1.998157570
> [81] 0.392193637 0.296714088 0.882998642 -0.429536165 0.231143614
> [86] 1.035526453 1.961756606 2.094884199 0.566998180 1.101794663
> [91] -0.144283435 -0.803213262 -1.692206658 -0.077911387 -1.601569929
> [96] -0.237983113 -0.001757424 -0.934857787 0.297604661 -1.518219533
> [ reached 'max' / getOption("max.print") -- omitted 900 entries ]
```

When no additional data are supplied, fitted values for the training sample are returned.

Predictions for new observations can be generated via the `newdata` argument.

```
new_data <- sample_data[1:5, -1]

predict(
  fit,
  newdata_X = new_data,
  newdata_Z = new_data
)
> [1] -0.1046202 -0.7666814 -1.7328684 0.3128017 -1.3462061
```

The prediction method also supports evaluating models at earlier boosting iterations.

```
predict(
  fit,
  newdata_X = new_data,
  newdata_Z = new_data,
  mstop = 25
)
> [1] -0.1447516 -0.2282335 -0.5555850 0.2301184 -0.3352459
```

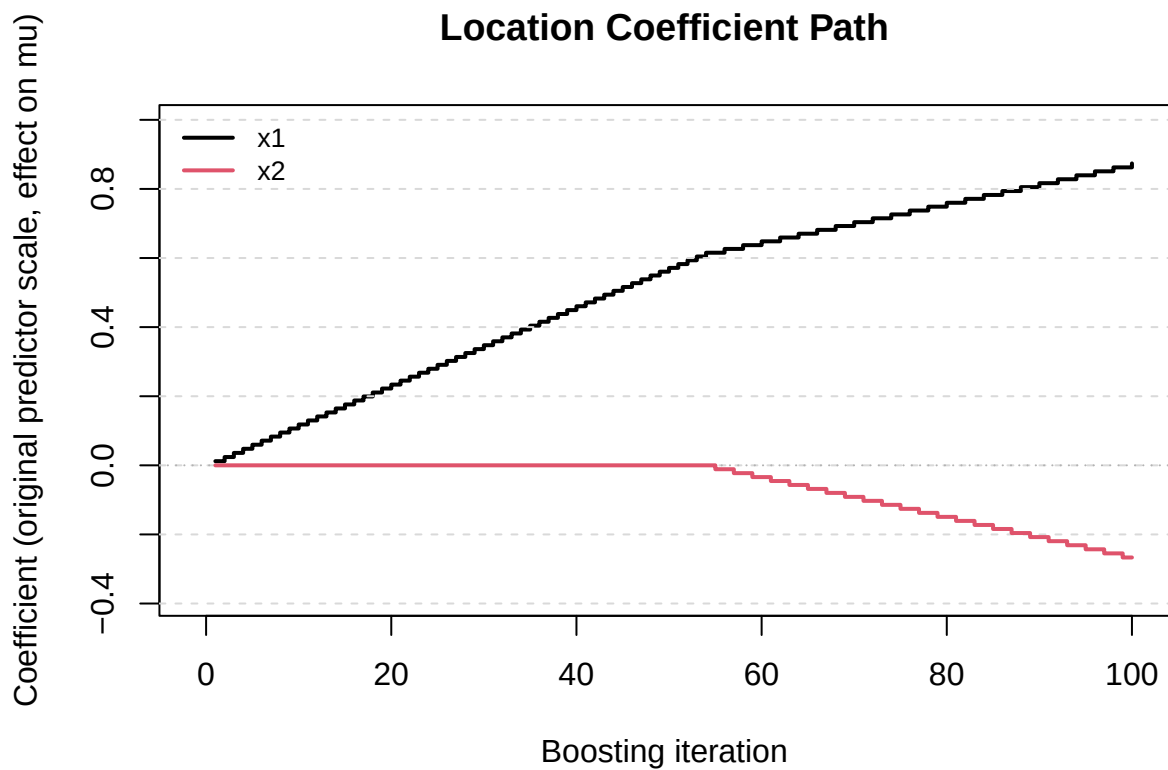
Note For in-sample early-stopped predictions, the model must have been fitted with `keep_path = TRUE`, which is the default when `keep_path` is unspecified.

Model Diagnostics and Visualization

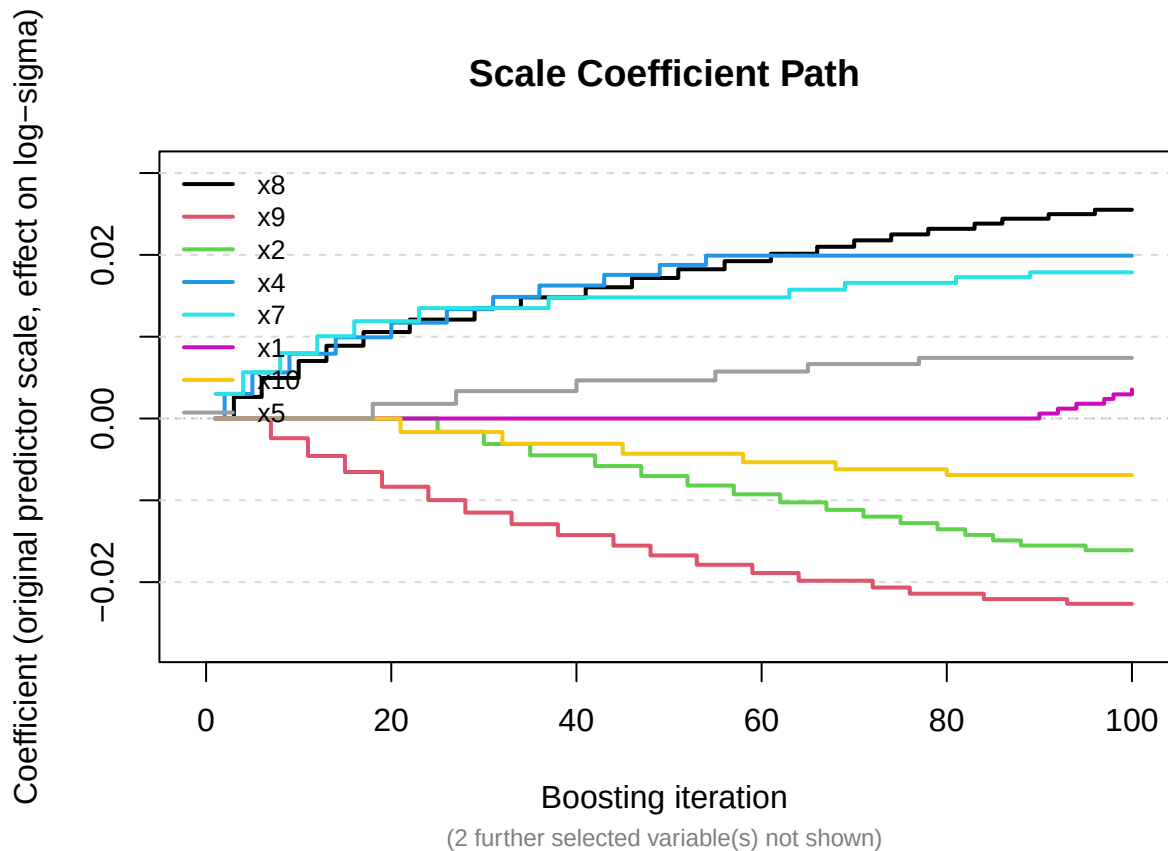
The package provides graphical diagnostics through the `plot()` method.

Coefficient trajectories across boosting iterations can be visualized using `plot()` method by specifying `type = "path"`.

```
# For Location
plot(
  fit,
  type = "path",
  submodel = "mu",
  top_n = 8,
  scale = "original"
)
```



```
# And for scale
plot(
  fit,
  type = "path",
  submodel = "sigma",
  top_n = 8,
  scale = "original"
)
```

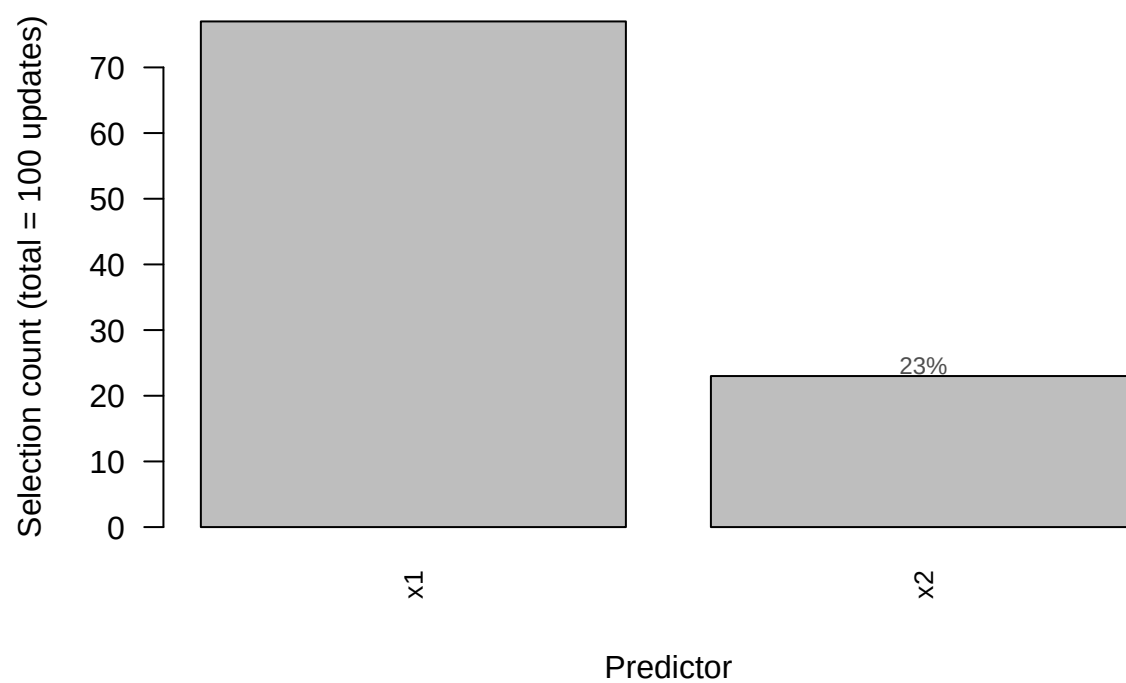


By default, the eight coefficients with the highest selection frequencies are displayed. Coefficients may be shown either on the original (default) predictor scale or on the standardized training scale.

Variable selection frequencies can be inspected by setting `type = "frequency"`.

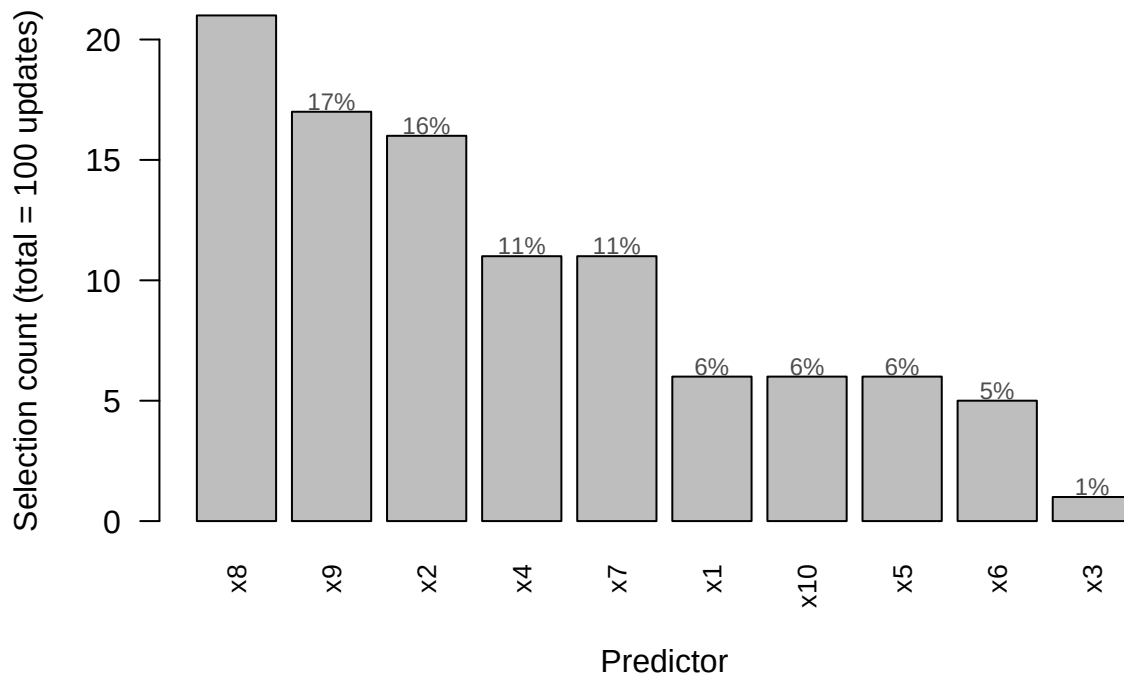
```
# For Location
plot(
  fit,
  type = "frequency",
  submodel = "mu",
  top_n = 8
)
```

Location Variable Selection Frequency



```
# And for scale  
plot(  
  fit,  
  type = "frequency",  
  submodel = "sigma",  
  top_n = 8  
)
```

Scale Variable Selection Frequency



This visualization summarizes how often each predictor was selected during the boosting process and provides a simple measure of variable importance. Again the number of Predictors displayed can be adjusted via `top_n`.

Note To use the `plot()` diagnostics the model must be fitted with `keep_path = TRUE`, which is the default.

Cross-Validation for Selecting the Stopping Iteration

A key tuning parameter in component-wise boosting is the number of boosting iterations, denoted by `mstop`. While increasing the number of iterations typically decreases the training error, excessively large values may lead to overfitting. Consequently, early stopping serves as the primary regularization mechanism in statistical boosting, encouraging sparsity in the selected variables.

A common strategy for selecting an appropriate stopping iteration is cross-validation. The data is repeatedly split into training and validation subsets. Models are fitted on the training folds and evaluated on the corresponding validation folds. The value of `mstop` that minimizes the estimated out-of-sample prediction error is then selected.

The function `cv_boost_gaussian()` implements k-fold cross-validation for Gaussian component-wise boosting models.

```
K = 5 # Number of folds during cross-validation
```

```
cv_fit <- cv_boost_gaussian(  
  X = sample_data[, -1],  
  Z = sample_data[, -1],  
  y = sample_data$y,  
  k = K,  
  mstop = 1000,
```

```

nu_mu = 0.05,
nu_sigma = 0.05,
seed = 27092001
)

```

Inspecting Cross-Validation Results

The resulting object contains the selected variables in the optimal model, the optimal `mstop`, and a final model refitted on the complete dataset using the optimal value of `mstop`.

```

cv_fit
>
> Call:
> cv_boost_gaussian(X = sample_data[, -1], Z = sample_data[, -1],
>   y = sample_data$y, k = K, mstop = 1000, nu_mu = 0.05, nu_sigma = 0.05,
>   seed = 27092001)
>
> Optimal mstop: 518
> Selected variables (location):
> [1] "x1" "x2" "x3" "x10" "x8" "x6" "x7" "x4" "x9"
> Selected variables (scale):
> [1] "x7" "x4" "x8" "x9" "x5" "x10" "x2" "x6" "x1" "x3"

```

A more detailed summary can be obtained via:

```

summary(cv_fit)
>
> Call:
> cv_boost_gaussian(X = sample_data[, -1], Z = sample_data[, -1],
>   y = sample_data$y, k = K, mstop = 1000, nu_mu = 0.05, nu_sigma = 0.05,
>   seed = 27092001)
>
> Optimal mstop: 518
>
> Residuals:
>   Min.   1st Qu.   Median     Mean   3rd Qu.    Max.
> -5.51131 -0.60567 -0.02708  0.01038  0.63430  4.41289
>
> Coefficients (location):
>      x1      x2      x3      x4      x5      x6
> 1.969898242 -1.506276221 0.968615009 -0.004856946 0.000000000 0.023391941
>      x7      x8      x9      x10
> -0.012128777 0.026737107 -0.002185538 0.045933033
>
> Coefficients (scale):
>      x1      x2      x3      x4      x5      x6
> 0.223965661 -0.172672002 0.090270671 -0.030401931 0.007410883 -0.047843929
>      x7      x8      x9      x10
> 0.014130362 0.017560190 0.014325333 0.035424595

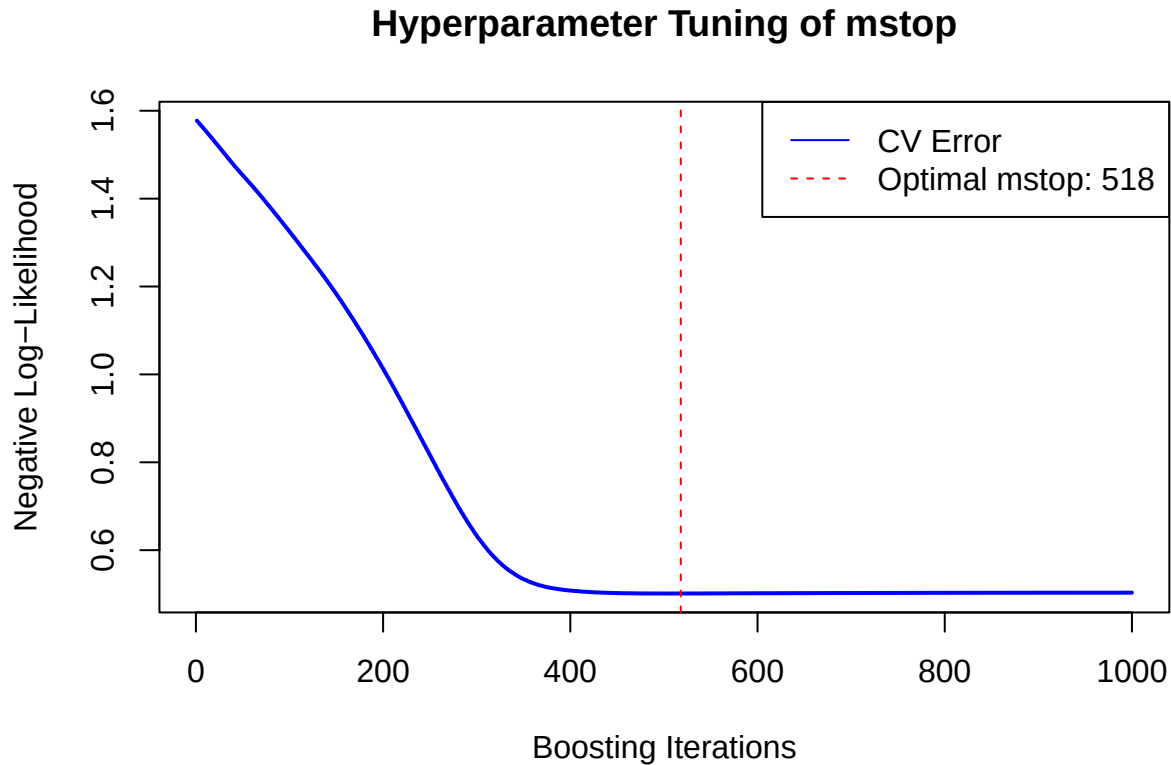
```

This gives additional information on the Residuals and the Coefficients.

Visualizing the Cross-Validation Curve

The estimated validation error can be visualized using the plot method.

```
plot(cv_fit)
```



The blue curve represents the mean validation error across folds as a function of the boosting iteration. The vertical dashed line indicates the selected stopping iteration minimizing the estimated prediction error.

Prediction Using the Cross-Validated Model

Predictions from a cross-validated model can be generated directly via `predict()`.

```
predict(cv_fit)
> [1]  1.355327760 -1.563849426 -4.387387488 -0.627867608 -3.934943622
> [6] -3.894995769  1.723928238  2.005720361  0.736362585 -3.145335757
> [11] -6.521942683  1.783127168  4.281041204 -2.609455646 -4.294229266
> [16] -0.634182388 -3.647317043  0.585193297  1.000709902  3.610293001
> [21] -4.000895718  0.563293692 -0.956718078 -1.986156339  1.694686608
> [26]  1.079742617 -0.878437856  2.973187752 -3.975046685 -4.593397783
> [31]  0.327742437 -2.214449479  0.914064084  1.752272085  1.014212730
> [36] -1.667612843  2.733998474 -3.433904635  7.416645442  1.019643737
> [41]  3.443940301  1.555308078 -0.336166605 -0.075854671 -0.038592996
> [46] -0.803384732 -0.864296161 -1.166980098  2.159685969 -0.699782595
> [51] -4.423136207  1.176266532  4.908988894  3.286901467  5.161941698
> [56] -1.759077399 -0.960557414  1.463070323 -0.583894234  2.925337277
> [61] -7.507625170  3.221837006 -0.527558704 -3.545987272 -2.974170769
```

```

> [66] -3.972646854  3.547514430 -2.232810846 -0.619208944 -0.934682242
> [71] -2.419550770  1.008476901 -0.006877818 -3.778945699 -0.553241642
> [76]  2.636707799 -0.511322047  1.792649499 -3.756831598 -6.966163277
> [81]  2.995985570  0.071127746  1.293471225 -4.406515991  0.877205805
> [86]  1.714819750  3.020181032  6.834510321  0.933153144  0.390209513
> [91] -2.110468177 -3.452402774 -2.843478967  0.636975502 -2.757458008
> [96] -2.245408331  0.683258113 -3.717653287  0.091122067 -2.293333194
> [ reached 'max' / getOption("max.print") -- omitted 900 entries ]

```

When no new data is supplied, fitted values from the refitted model are returned.

Predictions for new observations can be obtained by specifying newdata.

```

new_data <- sample_data[1:5, -1]

predict(
  cv_fit,
  newdata_X = new_data,
  newdata_Z = new_data
)
> [1]  1.3553278 -1.5638494 -4.3873875 -0.6278676 -3.9349436

```

Internally, predictions are delegated to the final boosting model fitted using the optimal mstop value.

Hyperparameter Grid Search

In some situations, both the stopping iteration mstop and the learning rate nu may be treated as tuning parameters. The function cv_boost_grid() evaluates combinations of candidate values using repeated cross-validation.

```

grid_fit <- cv_boost_grid(
  X = sample_data[, -1],
  Z = sample_data[, -1],
  y = sample_data$y,
  k = 5,
  mstop_grid = c(50, 100, 200),
  nu_mu_grid = c(0.01, 0.1, 0.3),
  nu_sigma_grid = c(0.01, 0.1, 0.3),
  seed = 27092001
)

grid_fit
>
> Call:
> cv_boost_grid(X = sample_data[, -1], Z = sample_data[, -1], y = sample_data$y,
>   k = 5, mstop_grid = c(50, 100, 200), nu_mu_grid = c(0.01,
>   0.1, 0.3), nu_sigma_grid = c(0.01, 0.1, 0.3), seed = 27092001)
>
> Best combination:
>   mstop nu_mu nu_sigma cv_error
>  26    100   0.3      0.3 0.5010698

```

The returned object contains the formula call and the best combination of hyperparameters.

Concluding Remarks

This vignette introduced the basic workflow for Gaussian component-wise boosting, including model fitting, prediction, visualization, and cross-validation-based hyperparameter tuning.